

Refactoring. to eclipse collections

Making Your Java Streams
Leaner, Meaner, and Cleaner

Donald Raab, Vladimir Zakharov

dev2next • Colorado Springs, CO • September 29 - October 2, 2025

<https://github.com/vmzakharov/refactor-to-ec-remake>

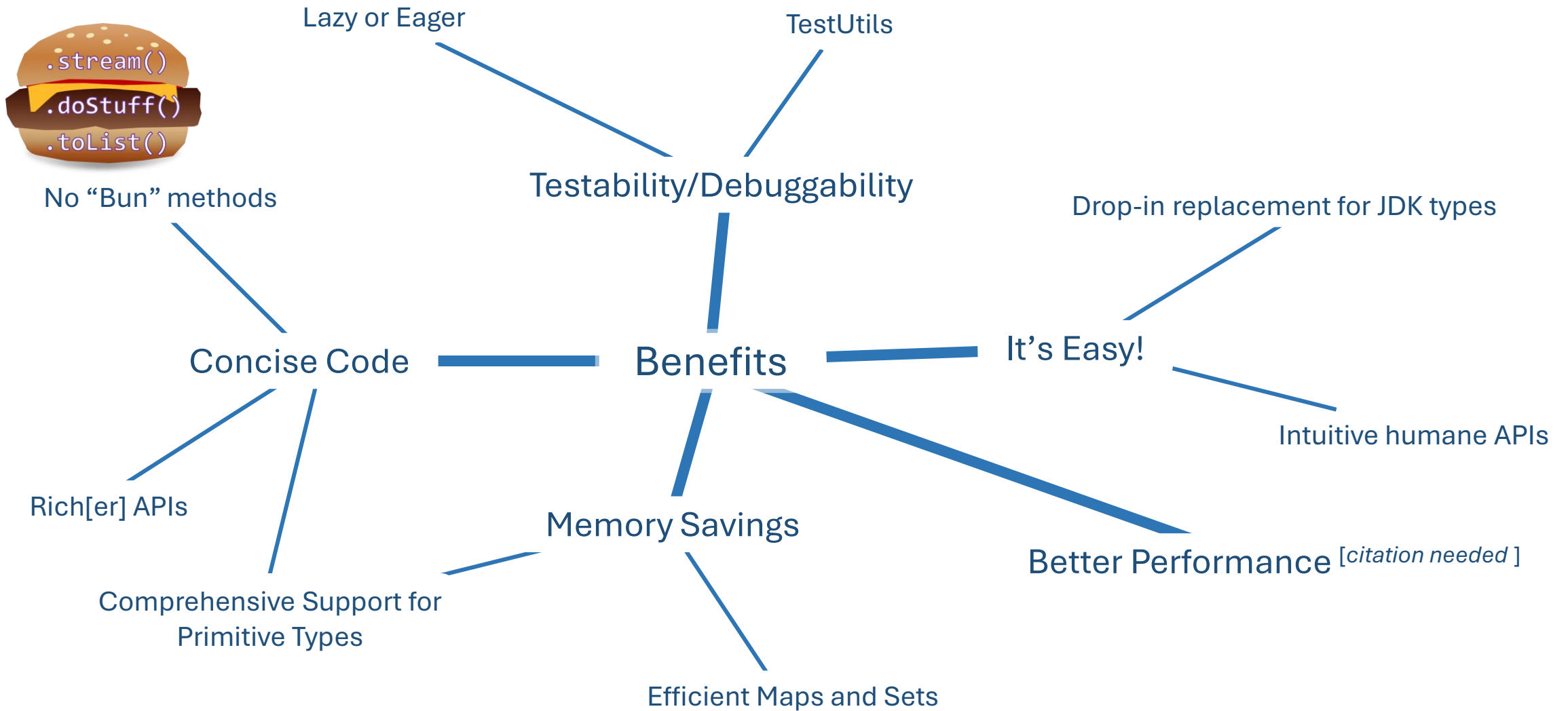


Introduction

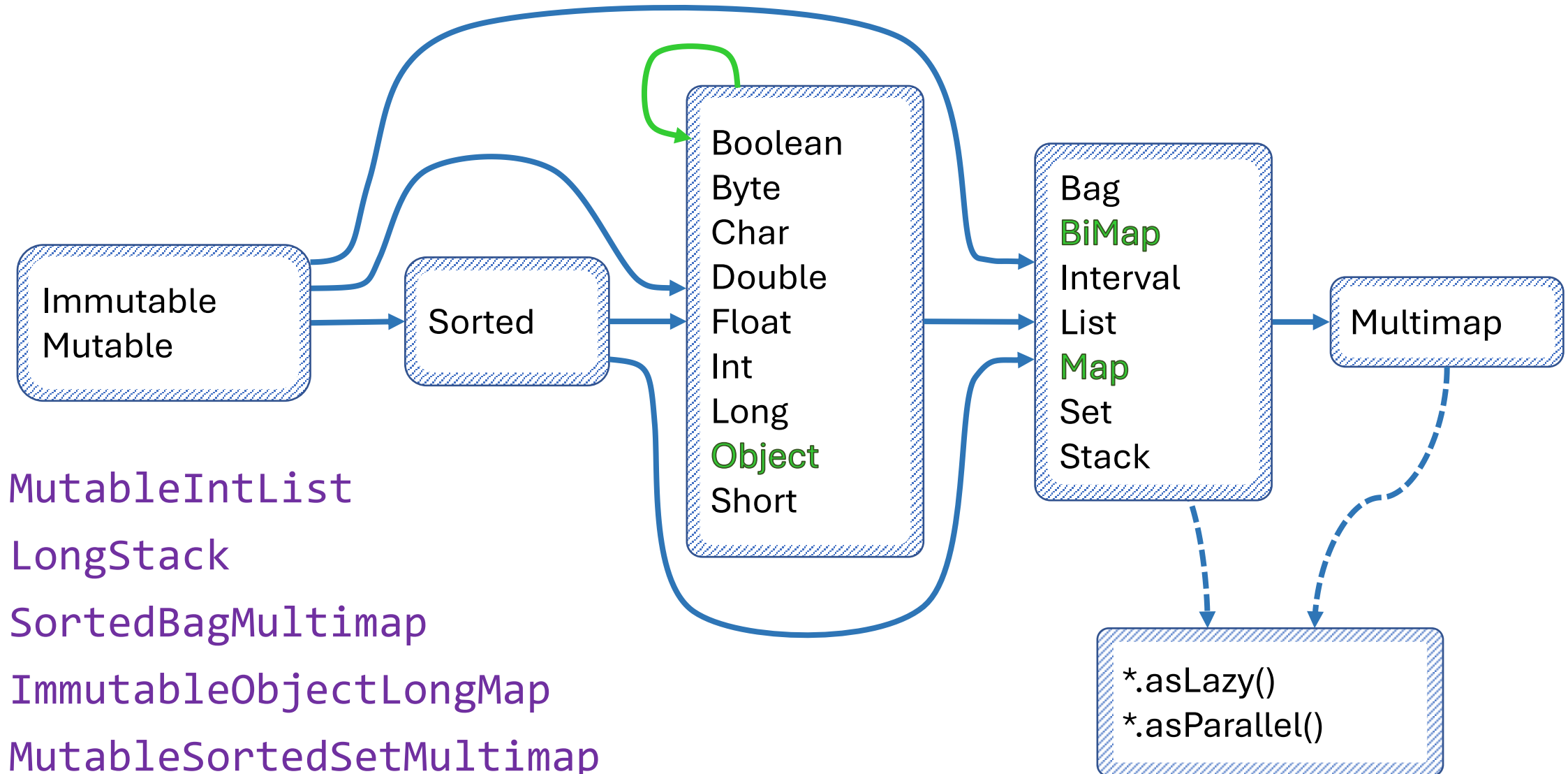
- What is Eclipse Collections?
 - Feature rich, memory efficient Java Collections framework
- History
 - Eclipse Collections started off as an internal collections framework named Caramel at Goldman Sachs in 2004
 - In 2012, it was open sourced to GitHub as a project called [GS Collections](#)
 - GS Collections was migrated to the Eclipse Foundation, re-branded as [Eclipse Collections](#) in 2015
 - Eclipse Collections 12.0 (Java 11+) & 13.0 (Java 17+) released in June 2025
- Learn Eclipse Collections with [Kata](#)
- Eclipse Collections is [open for contributions!](#)



Why Refactor to EC?



Any Types You Need



Instantiate Them Using Factories

Payload Type*	Container Type	Mutability	Multimap	Initialized	Lazy
Boolean	Bags	.mutable	.bag	.empty() .of() .with()	.asLazy()
Byte	BiMaps	.immutable	.list	.of(one) .with(one)	
Char	Lists	.fixedSize	.set	...	
Double	Maps		.sortedSet	.of(one,...,ten)	
Float	Multimaps			.with(one,...,ten)	
Int	Sets			.of(... elements)	
Long	SortedBags			.with(... elements)	
Object	SortedMaps			.ofAll(Iterable)	
Short	SortedSets			.withAll(Iterable)	
	Stacks				

* Pick two
for maps

ImmutableLongStack

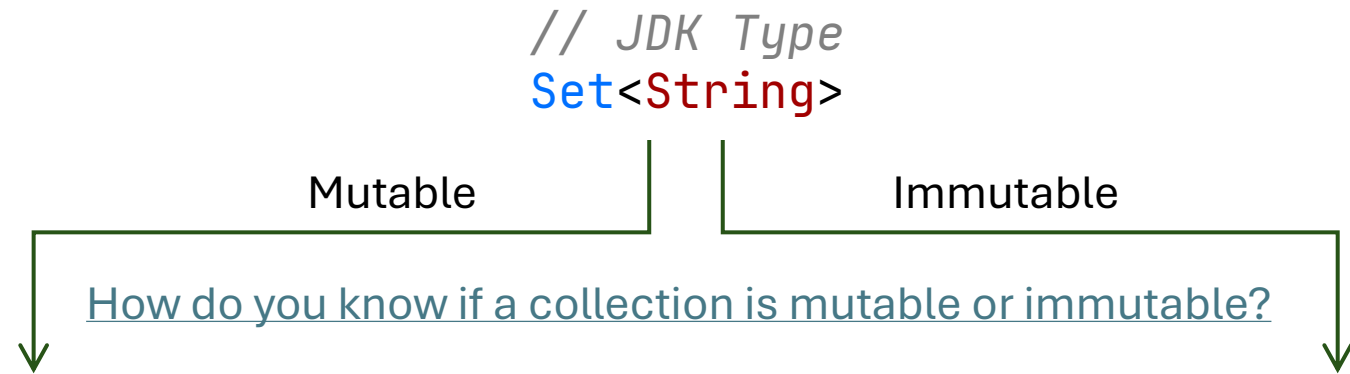


LongStacks.*immutable*.with(1, 2, 3).asLazy()

LazyLongIterable



Evolving from JDK to EC Factories and Types



// JDK Type
`Set<String>`

// JDK Implementation
`new HashSet<>(List.of("1", "2", "3"))`

// JDK Type
`Set<String>`

// JDK Implementation
`Set.of("1", "2", "3")`

// JDK Type
`Set<String>`

// EC Implementation
`Sets.mutable.of("1", "2", "3")`

// JDK Type
`Set<String>`

// EC Implementation
`Sets.immutable.of("1", "2", "3").castToSet()`

// EC Type
`MutableSet<String>`

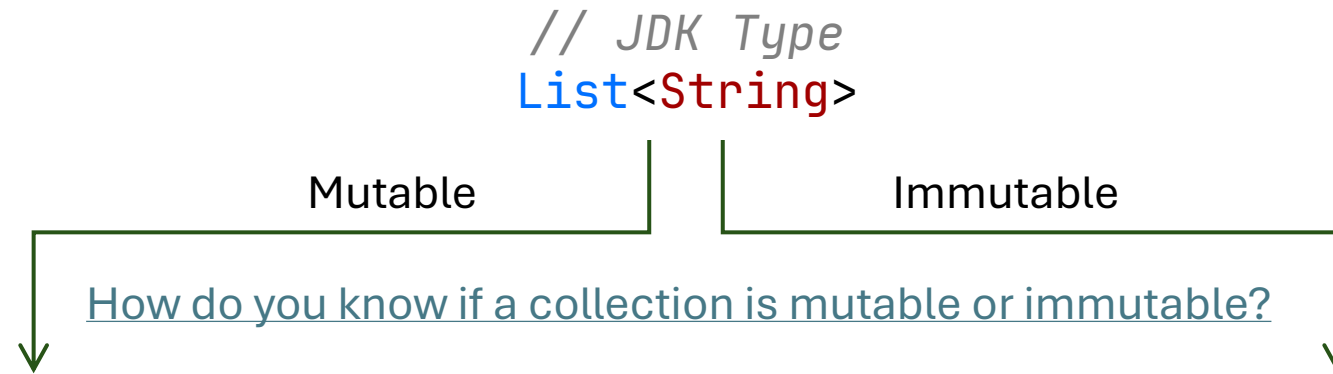
// EC Implementation
`Sets.mutable.of("1", "2", "3")`

// EC Type
`ImmutableSet<String>`

// EC Implementation
`Sets.immutable.of("1", "2", "3")`



Evolving from JDK to EC Factories and Types



// JDK Type
`List<String>`

// JDK Implementation
`new ArrayList<>(List.of("1", "2", "3"))`

// JDK Type
`List<String>`

// EC Implementation
`Lists.mutable.of("1", "2", "3")`

// EC Type
`MutableList<String>`

// EC Implementation
`Lists.mutable.of("1", "2", "3")`

// JDK Type
`List<String>`

// JDK Implementation
`List.of("1", "2", "3")`

// JDK Type
`List<String>`

// EC Implementation
`Lists.immutable.of("1", "2", "3").castToList()`

// EC Type
`ImmutableList<String>`

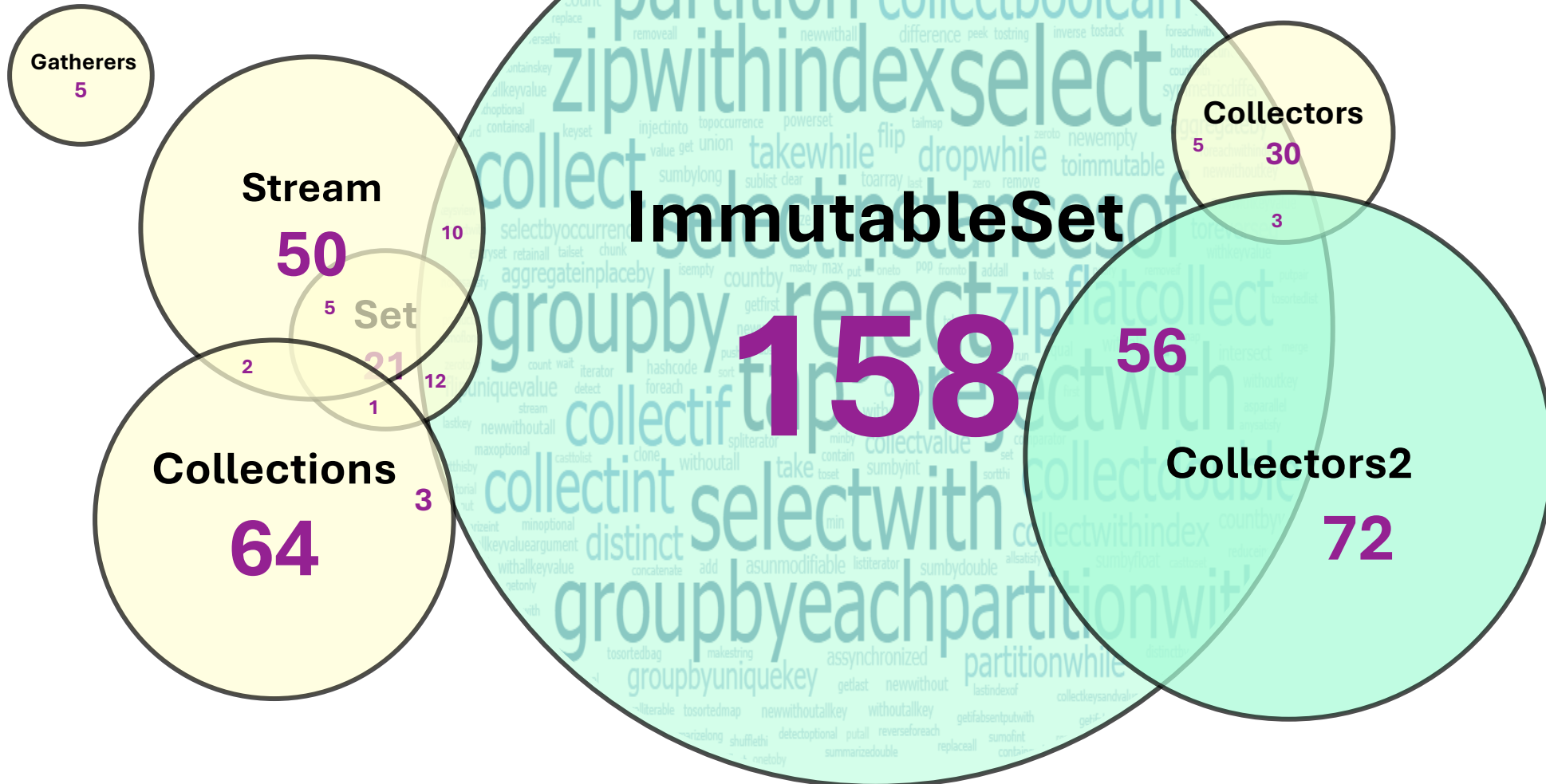
// EC Implementation
`Lists.immutable.of("1", "2", "3")`



Set/Stream vs. ImmutableSet

- Java Collections / Streams
- Eclipse Collections

Metric - Number of Non-overloaded Methods



Methods by Category - Highlights

transforming

collect[With]
collect[Boolean,Byte,Char,Double,Float,Int,Long,Short]
collectIf
collectKeysAndValues
collectValues
collectWithIndex
collectWithOccurrences
flatCollect

grouping

groupBy
groupByEach
groupByUniqueKey
sumBy[Double,Float,Int,Long]
sumOf[Double,Float,Int,Long]
aggregateBy
aggregateInPlaceBy

wrapping

asLazy
asParallel
asReversed
asSynchronized
asUnmodifiable

finding

detect[With]
detect[With]IfNone
detect[With]Optional
max[By]
min[By]

converting

toArray
toBag
toImmutable
toList
toMap
toReversed
toSet
toSortedBag[By]
toSortedList[By]
toSortedMap
toSortedSet
toSortedSet[By]
toStack
toString

filtering

select[With]
selectByOccurrences
selectInstancesOf
reject[With]
partition[With]
partitionWhile

testing

allSatisfy[With]
anySatisfy[With]
noneSatisfy[With]
notEmpty
isEmpty



Simulating Method Categories in IntelliJ

- Using **Custom Code Folding Regions**

eclipse-collections-api-13.0.0-sources.jar > org > eclipse > collections > api >

Structure

- RichIterable
 - [Category: Iterating]
 - [Category: Counting]
 - [Category: Testing]
 - [Category: Finding]
 - [Category: Filtering]
 - [Category: Transforming]
 - [Category: Grouping]
 - groupBy(Function<? super T, ? extends V>): Multimap<V, T>
 - groupBy(Function<? super T, ? extends V>, R): R
 - groupByEach(Function<? super T, ? extends Iterable<V>>): Multimap<V, T>
 - groupByEach(Function<? super T, ? extends Iterable<V>>, R): R
 - groupByUniqueKey(Function<? super T, ? extends V>): Map<V, T>
 - groupByUniqueKey(Function<? super T, ? extends V>, R): R
 - chunk(int): RichIterable<RichIterable<T>>
 - groupByAndCollect(Function<? super T, ? extends K>, Function<? super K, ? extends V>): Multimap<K, V>
 - [Category: Aggregating]
 - [Category: Converting]

```
//region [Category: Iterating]
```

```
@Override
```

```
default void forEach(Procedure<? super
```

```
/** The procedure is executed for each  
/UnnecessaryFullyQualifiedName/
```

```
void each(Procedure<? super T> proced
```

```
/** Executes the Procedure for each e  
RichIterable<T> tap(Procedure<? super
```

```
/** Returns a lazy (deferred) iterable  
LazyIterable<T> asLazy();
```

```
//endregion [Category: Iterating]
```



Let's Do It!

Memory: Methodology

Java Object Layout (JOL) - [OpenJDK Code Tools Project](#)

JOL (Java Object Layout) is the tiny toolbox to analyze object layout schemes in JVMs. JOL [is] much more accurate than other tools relying on heap dumps, specification assumptions, etc.

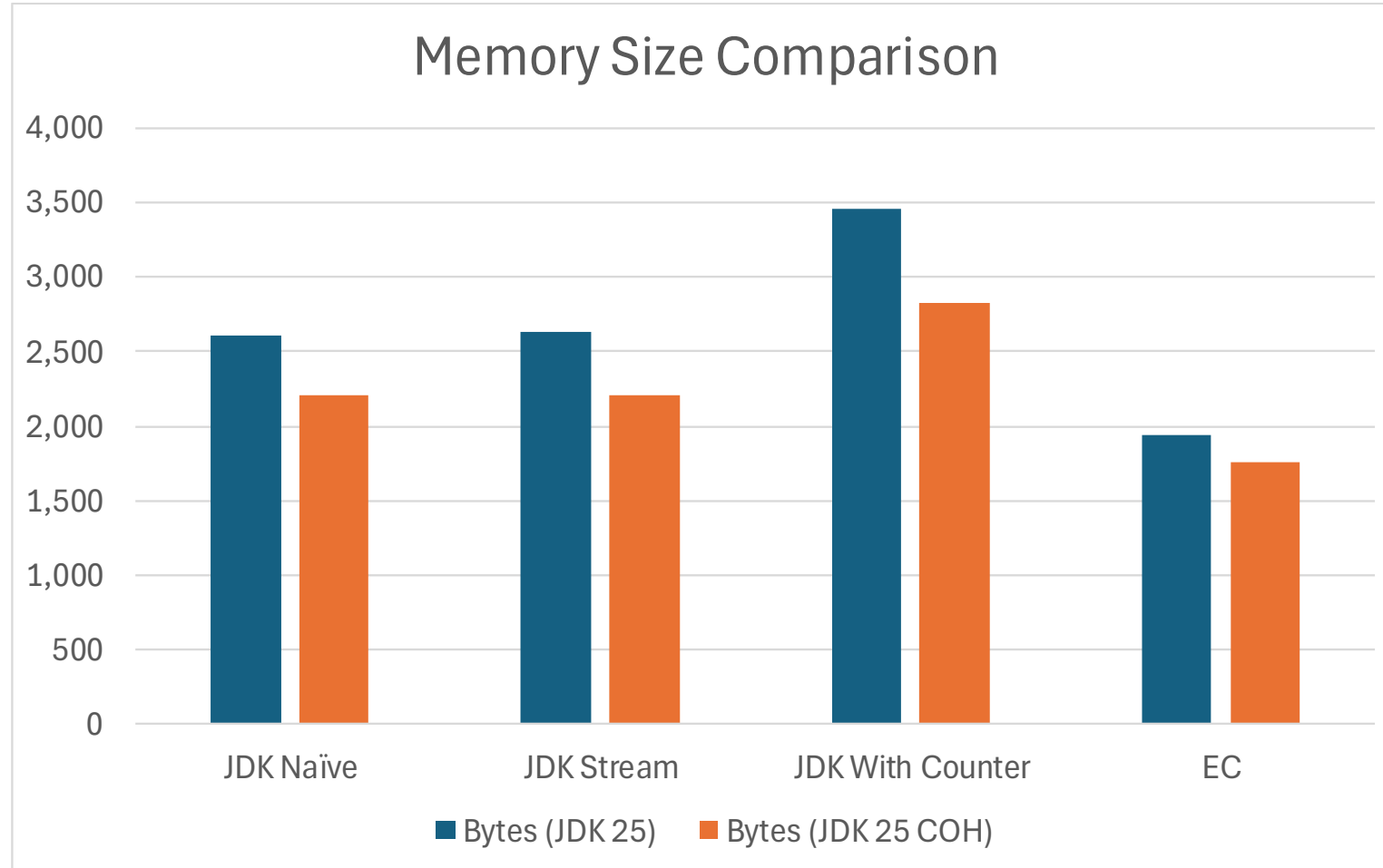
```
<dependency>
  <groupId>org.openjdk.jol</groupId>
  <artifactId>jol-core</artifactId>
  <version>0.17</version>
</dependency>
```

Further reading

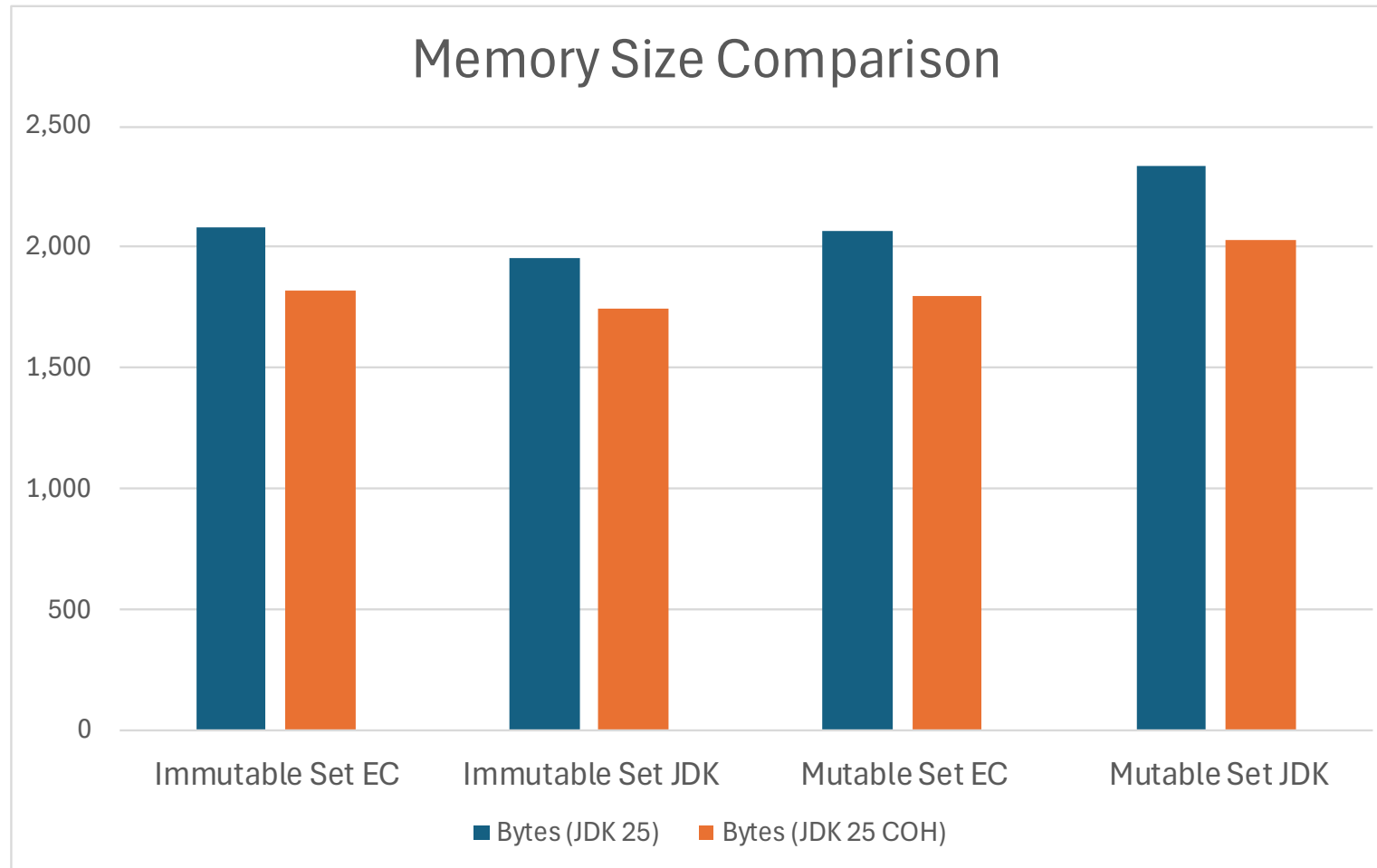
- Baeldung: [Memory Layout of Objects in Java](#)
- StackOverflow:
 - [Does Java Object Layout work with Java Records?](#)
 - [Does Java Object Layout \(JOL\) work with Java 25 and Compact Object Headers enabled?](#)



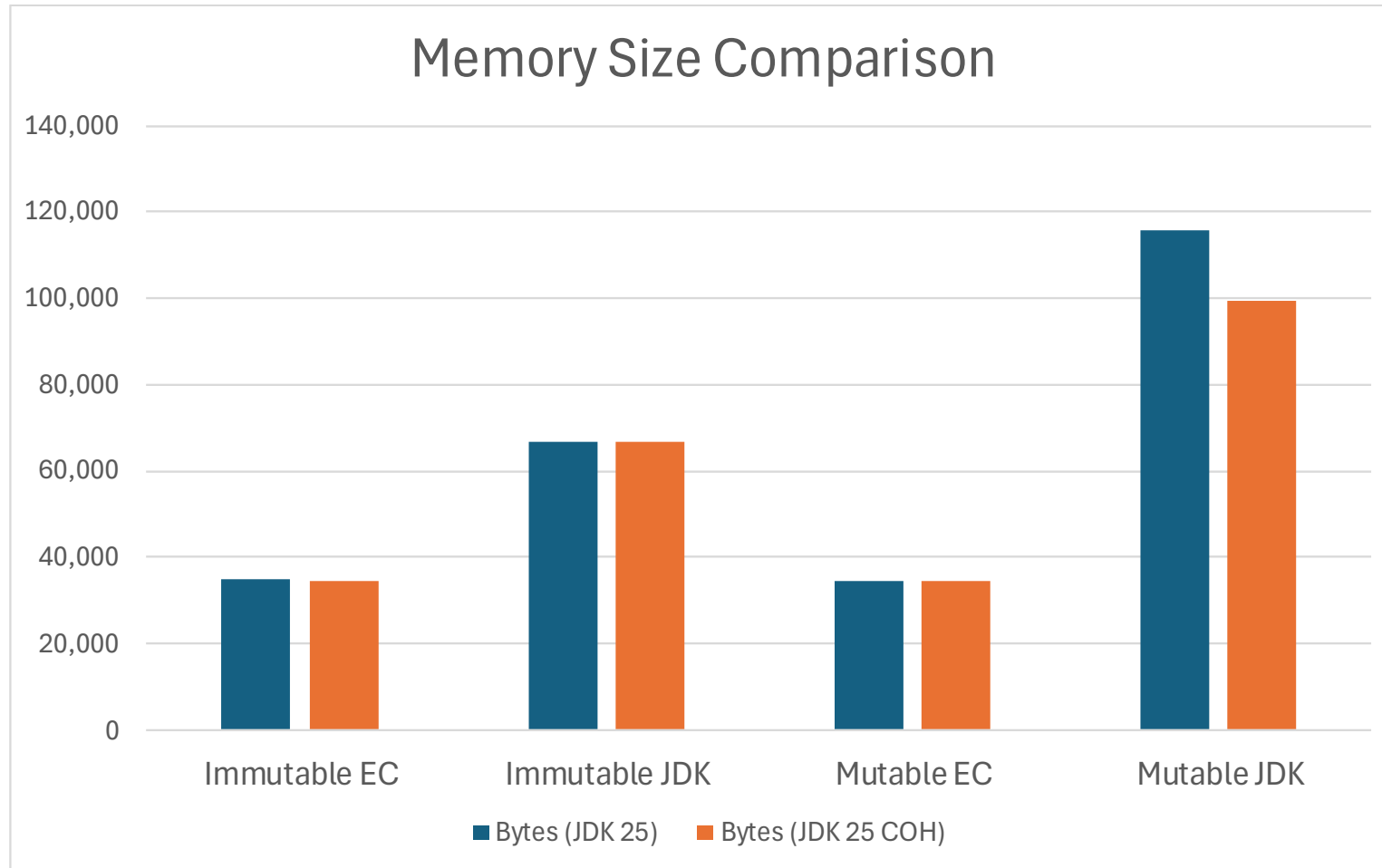
Memory: Word $\rightarrow$ Count Map



Memory: Generation Set

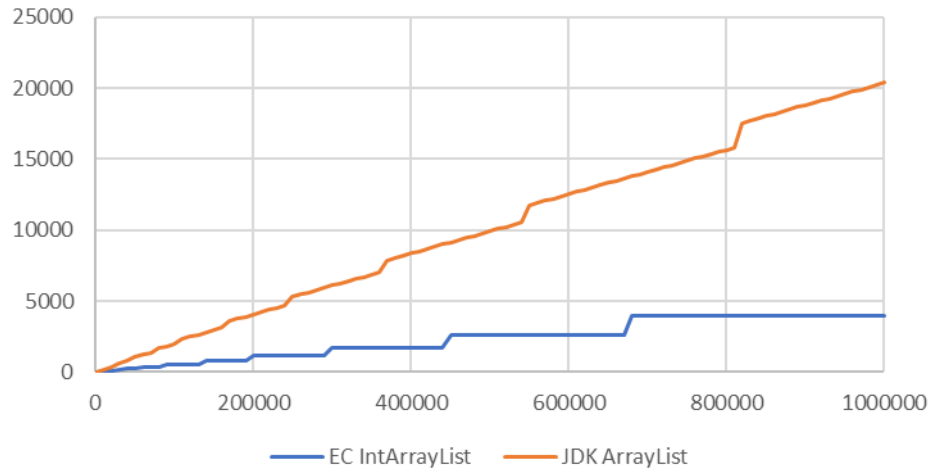


Memory: Year → Generation Map

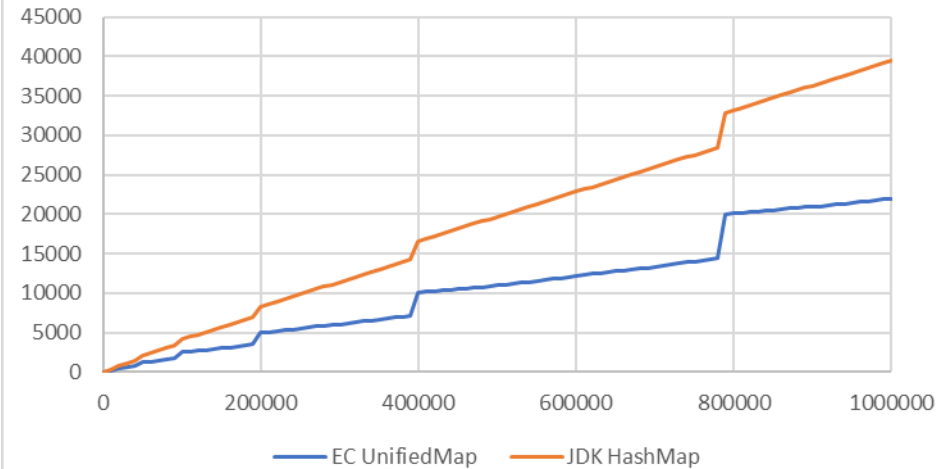


Container Overhead in KB vs. Size

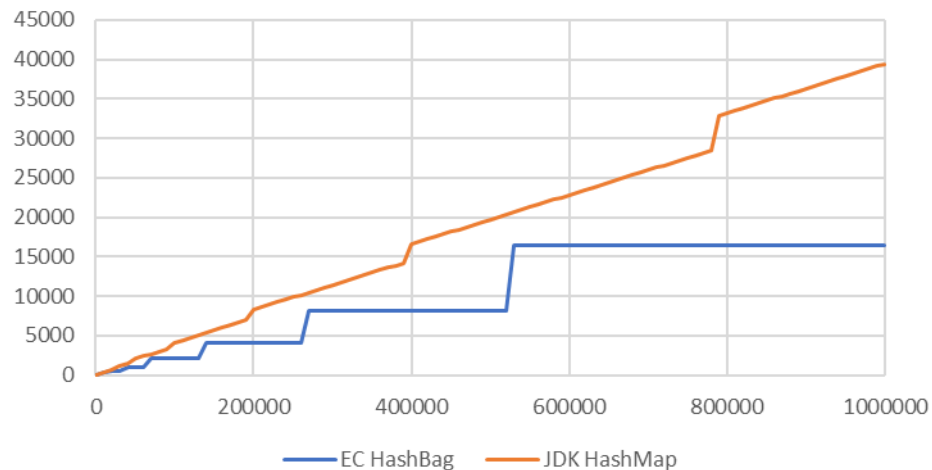
List of ints vs. List<Integer>



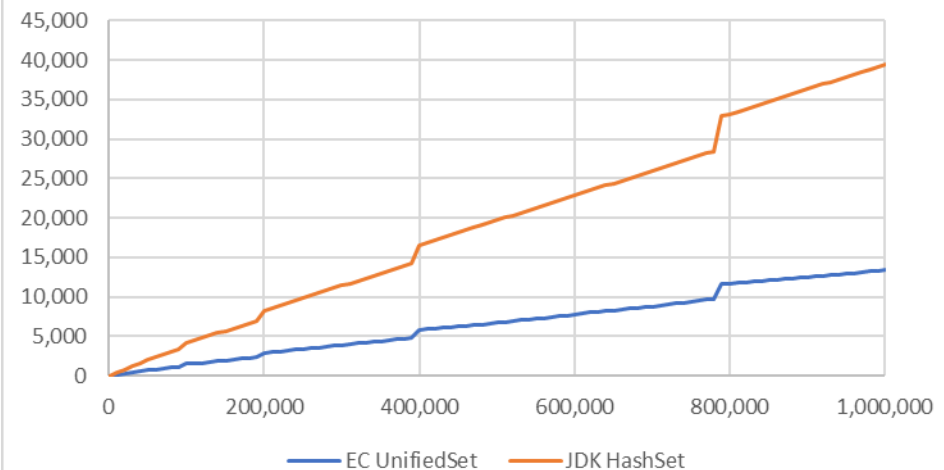
Map<String, String>



Bag<Strings> vs. Map<String, Integer>



Sets<String>



Eclipse Collections – What's in It for Me?

Features	Productivity	Performance	Thread Safety	Type Safety
Rich functional API with good symmetry	✓			
Memory Efficiency		✓		
Optimized Eager API		✓		
Primitive Collections	✓	✓		✓
Immutable Collections		✓	✓	
Lazy and Parallel Lazy API		✓		
Multimap, Bag, BiMap, Pool, Interval		✓		✓
Hashing Strategy Data Structures	✓	✓		
Multi-reader Data Structures		✓	✓	
Mutable and Immutable Collection Factories	✓			



Example: DataFrame-EC

...built on the solid foundation of Eclipse Collections types

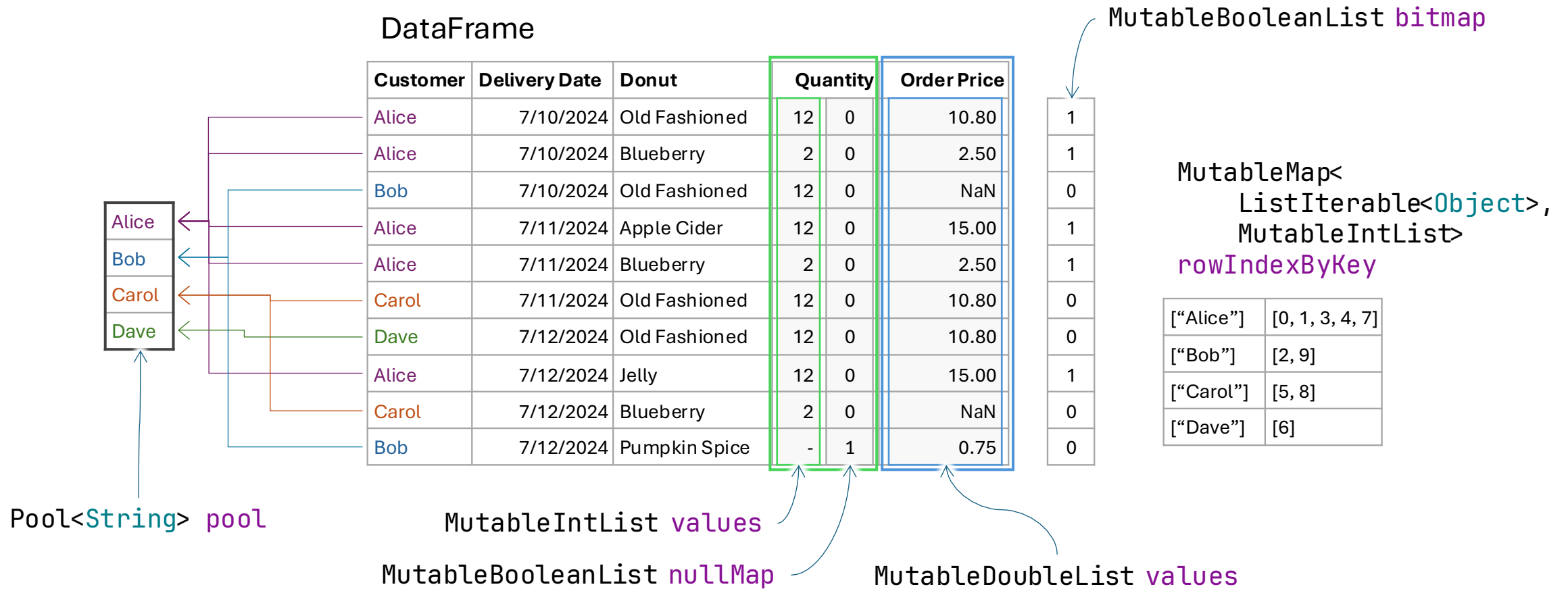
DataFrame

Customer	Delivery Date	Donut	Quantity	Order Price
Alice	7/10/2024	Old Fashioned	12	10.80
Alice	7/10/2024	Blueberry	2	2.50
Bob	7/10/2024	Old Fashioned	12	null
Alice	7/11/2024	Apple Cider	12	15.00
Alice	7/11/2024	Blueberry	2	2.50
Carol	7/11/2024	Old Fashioned	12	10.80
Dave	7/12/2024	Old Fashioned	12	10.80
Alice	7/12/2024	Jelly	12	15.00
Carol	7/12/2024	Blueberry	2	null
Bob	7/12/2024	Pumpkin Spice	null	0.75



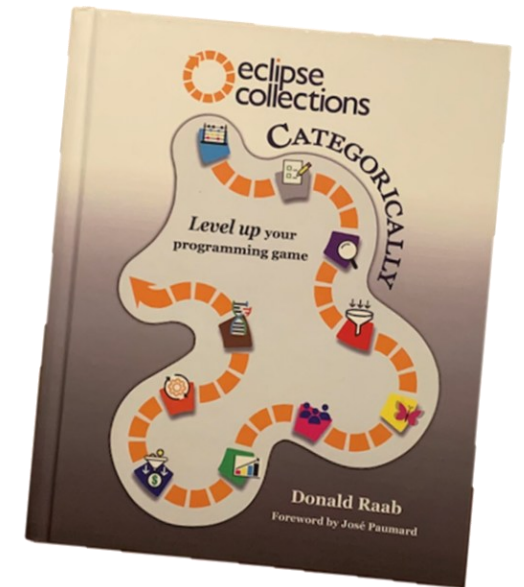
Example: DataFrame-EC

...the foundation, and walls, and joints, and studs, and...



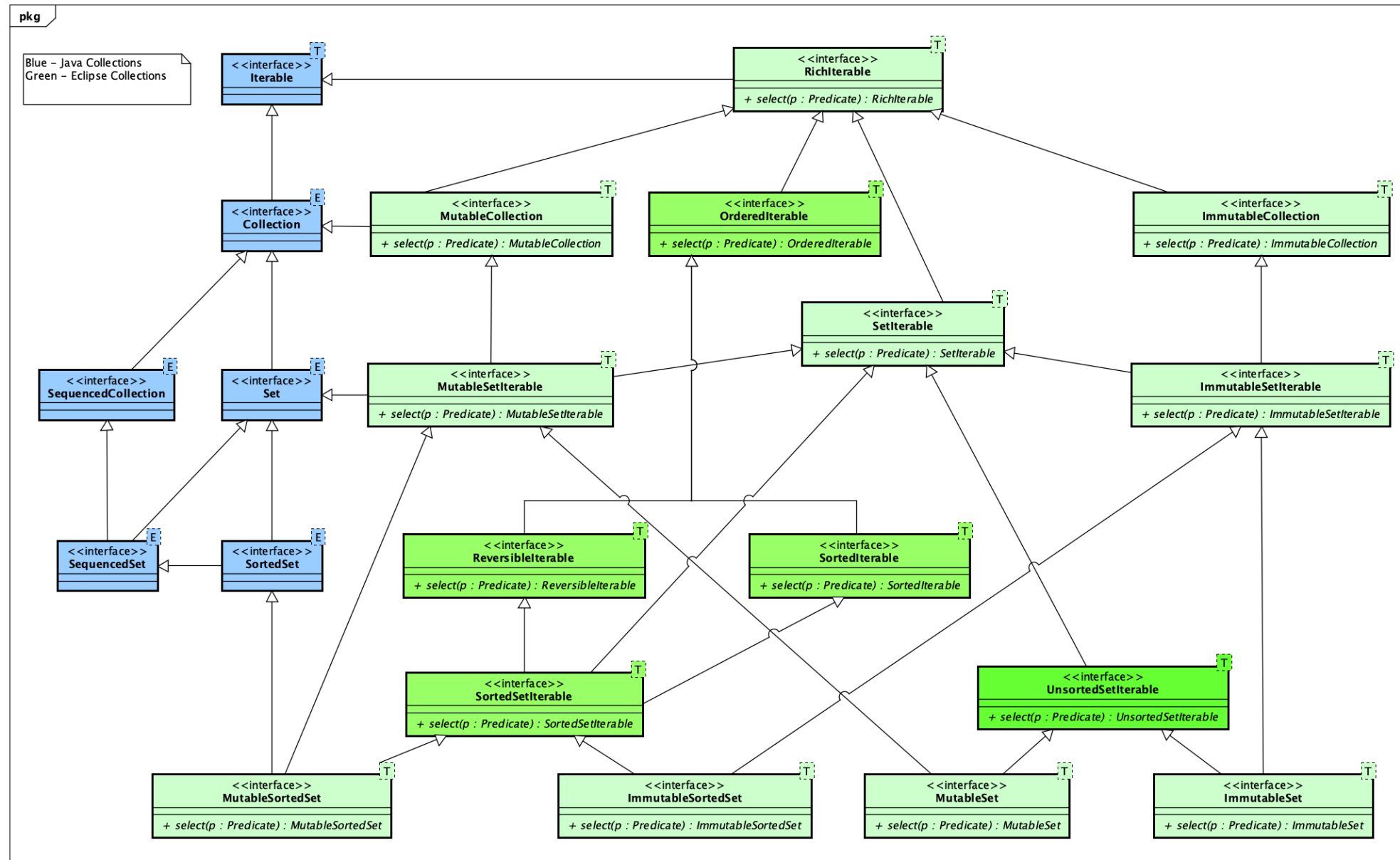
Takeaways

- Expect more from your collections
- Memory is an expensive resource – use it wisely
- Eclipse Collections – more features with less waste
- Intention revealing code – easy to write, easy to read!
- Many resources available
 - Source code w/ Reference Guide in AsciiDoc
 - <https://github.com/eclipse-collections/eclipse-collections>
 - Code katas
 - <https://github.com/eclipse-collections/eclipse-collections-kata>
 - Book: Eclipse Collections Categorically



Appendix

Eclipse Collections Set Type Hierarchy



Non-JDK Collection Types in EC

